Two-pointer, hash map, sliding window

Week 3, Lecture 2 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026

What we will cover

- The hash-map pattern: turning $O(n^2)$ into $O(n)$ by caching lookups
- Two pointers on sorted input: eliminating the inner loop
- Sliding window: maintaining a running invariant across a contiguous subarray
- How to decide which pattern fits a problem you have not seen before

Part 1: the hash-map pattern

The core trade

- Brute force on “does X exist in this collection?” is O(n) per query
- Nested loops that repeat this query are $O(n^2)$ total
- A hash map caches the answer: lookup drops to O(1) amortized
- Trade: O(n) extra space. Almost always worth it in an interview

Two Sum: the canonical case

- Problem: find two indices whose values sum to a target
- Brute force: try every pair. O(n²) time, O(1) space
- Hash-map version: for each element, check whether (target - element) is already in the map

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i
```

One pass. O(n) time, O(n) space.

When to reach for the hash map

- You need to check membership or count frequency across elements
- You are doing a repeated lookup inside a loop (nested or not)
- The problem involves pairs, grouping, or detecting duplicates
- NeetCode Roadmap: Arrays & Hashing is the first pattern because it unlocks everything else

Part 2: two pointers

The entry condition

- Two pointers are valid when the input is sorted, or when the problem has a symmetric search space
- One pointer starts at the left, one at the right; they move toward each other
- At each step you decide which pointer to move based on whether the current sum is too high or too low
- The inner loop disappears because each pointer moves at most n times

Two Sum II: sorted array

- Input is sorted ascending; find two numbers that sum to a target
- Brute force: $O(n^2)$
- Two-pointer version:

```
def two_sum_sorted(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        s = numbers[left] + numbers[right]
        if s == target:
            return [left + 1, right + 1]
        elif s < target:
            left += 1
        else:
            right -= 1
```

$O(n)$ time, $O(1)$ space. No hash map needed because sort order does the work.

The O(n) argument for two pointers

- Each pointer starts at one end and never reverses
- Total moves across both pointers: at most n
- So the loop runs at most n times total, not n^2 times
- Compare with nested loops: inner loop runs n times for each outer step

When sort order is available, use it. It is free information.

Part 3: sliding window

What a sliding window is

- A contiguous subarray (or substring) whose left and right boundaries move
- You maintain an invariant: a property that must hold for the window to be valid
- When the window is valid, expand it (move right boundary out)
- When the window is invalid, shrink it (move left boundary in)
- One pass through the array: O(n)

Longest substring without repeating characters

- Invariant: no character appears more than once in the window
- Expand: move right, add the character to a set
- Shrink: move left, remove the character, until the invariant holds again

```
def length_of_longest_substring(s):
    chars = set()
    left = 0
    result = 0
    for right in range(len(s)):
        while s[right] in chars:
            chars.remove(s[left])
            left += 1
        chars.add(s[right])
        result = max(result, right - left + 1)
    return result
```


O(n) time, O(k) space where k is the character set size.

Writing the invariant before coding

- The invariant is your contract with the problem
- Write it in one sentence before you write any code: “the window contains no duplicate characters”
- Every time you move a boundary, ask: does the invariant still hold?
- Coding without naming the invariant produces solutions that almost work

Part 4: pattern recognition under time pressure

The recognition decision tree

- Does the problem involve membership, frequency, or grouping? Consider hash map
- Is the input sorted, or does the problem compare two elements from opposite ends? Consider two pointers
- Does the problem ask about a contiguous subarray or substring with a running constraint? Consider sliding window
- Start with the brute force, name its complexity, then ask which of the three patterns eliminates the bottleneck

What NeetCode’s method actually means in practice

- NeetCode (2020): “I wasn’t trying to memorize solutions. I was trying to understand the patterns well enough to derive any solution from scratch.”
- Pattern understanding = you can state why the pattern works, not just that it works
- Test yourself: explain Two Sum to someone who has not heard of hash maps. If you cannot, you have memorized, not understood.

> These 150 problems were hand-selected to cover every major pattern you'll see in a coding interview. If you can solve all of them, you're ready.

<footer>NeetCode, "NeetCode 150 Course," 2022</footer>

Take-aways

1. The hash-map pattern trades O(n) space for O(1) lookup, collapsing nested $O(n^2)$ loops to a single O(n) pass.
2. Two pointers are valid when sort order creates a monotone decision: moving one pointer always moves the sum in a predictable direction.
3. A sliding window maintains an invariant. Write the invariant before coding, and every boundary move follows from it.
4. Pattern recognition is a skill: train it by asking “which pattern eliminates the bottleneck?” after every brute-force solution.

What’s next

- **Reading:** Week 3 reading has full worked examples for all three patterns plus exercises (read before section if you have not)
- **Section (this week):** Pattern sprint 1, three timed 25-minute mediums with structured debrief
- **HW 1 due this week:** Resume and portfolio audit
- **HW 2 out this week:** 30-problem LeetCode pattern sprint
- **Next week (Week 4):** Recursion, binary trees, BFS, and DFS